The Epistemic Verification Gap: A Unified Theory of Transitive Trust Failures Across Procedural Supply Chains, Neural Parameter Spaces, and Cryptographic Hierarchies

Abstract— Modern computational architectures rely heavily on compositional abstractions, inadvertently cementing a pervasive topological vulnerability: perimeter-based security models colloquially described as "hard on the outside, soft and squishy on the inside." Once an artifact—whether a modular software dependency, a neural network weight matrix, or a cryptographic key—bypasses initial perimeter provenance validation, it is granted implicit, over-privileged trust within the execution environment. This paper explores the structural isomorphism of epistemic and procedural crises in three domains: the Node Package Manager (NPM) supply chain, Artificial Intelligence (AI) model takeover via weight poisoning, and the Domain Name System Security Extensions (DNSSEC) key bootstrapping bottleneck. We formalize the "Epistemic Verification Gap," demonstrating that perimeter defenses validate only syntactic integrity, failing to bound semantic execution. We critically examine the ontology of neural networks—analyzing what high-dimensional model weights mathematically encode and how their safety can be verified amid polysemanticity. Finally, we propose a unified architectural paradigm shift toward Object-Capability (OCap) execution, mechanistic interpretability via Sparse Autoencoders (SAEs), and decentralized cryptographic provenance to mathematically harden the interior of computational systems.
Keywords— *Software Supply Chain, Ambient Authority, Mechanistic Interpretability, Large Language Models, DNSSEC, Transitive Trust, Epistemic Opacity, Object-Capability Models*.

I. Introduction: The Topological Fallacy of Perimeter Security

Historically, information security architectures have mirrored physical fortifications. Cheswick and Bellovin [1] famously described the traditional network security model as a "crunchy shell around a soft, chewy center." Perimeter defenses—such as Transport Layer Security (TLS) gateways, firewalls, and access control lists—fortify the external boundary, yet grant ambient, over-privileged trust to any payload that successfully bypasses initial validation.

In the era of hyper-composable architectures, this topological model mathematically collapses. Systems no longer execute monolithic, statically analyzed, internally developed binaries. Instead, they dynamically ingest execution logic, cryptographic roots, and billion-parameter neural abstractions from decentralized global networks.

In all three scenarios explored in this paper, systems cryptographically secure the *delivery mechanism* (the hard outside) but fail to interrogate the *behavioral reality* of the payload (the

squishy inside). Once the perimeter establishes Provenance, the execution context implicitly assumes Semantic Integrity. This paper argues that this assumption is a foundational systemic flaw, linking procedural supply chain attacks, AI model poisoning, and DNSSEC bootstrapping failures under a single theoretical framework.

II. Theoretical Framework: The Epistemic Verification Gap

The foundational vulnerability of transitive trust was formalized by Ken Thompson in his seminal 1984 Turing Award lecture, *Reflections on Trusting Trust* [2], which demonstrated that trusting code is inherently flawed if the compiler generating the code is compromised. Modern ecosystems have scaled Thompson's dilemma exponentially.

We formalize the **Epistemic Verification Gap**. Let $P(x)$ represent Provenance Verification (e.g., cryptographic hashes, digital signatures confirming authorship and transit integrity) and $S(x)$ represent Semantic Verification (mathematical proof of benign execution intent). The systemic pathology of modern computing is the axiomatic, yet false, operational assumption that:

$$ P(x) = \text{Valid} \implies S(x) = \text{Safe} $$

$P(x)$ asserts historical properties of an artifact, whereas $S(x)$ attempts to bound its future behavioral reality. Delivering cryptographically verified artifacts directly into highly permissive runtimes guarantees inevitable compromise.

III. Procedural Opacity: Transitive Collapse in the NPM Ecosystem

The Node Package Manager (NPM) ecosystem represents the apotheosis of the "squishy interior" in procedural software. Modern applications are not written; they are aggregated.

A. Directed Acyclic Graphs and Transitive Closure

An application's dependency tree can be formally modeled as a Directed Acyclic Graph (DAG) $G = (V, E)$, where V represents individual software packages and E represents dependency relations. While an engineering team may thoroughly audit their direct dependencies (V_{direct}), the execution environment blindly inherits the transitive closure of this graph ($V_{transitive}$). Research indicates that the average NPM package relies on roughly 80 transitive dependencies, authored by unverified pseudonymous actors [3].

B. The Exploitation of Ambient Authority

The NPM perimeter is rigorously hardened via TLS, registry multi-factor authentication, and cryptographic provenance frameworks like Sigstore [4]. However, attack vectors—such as Dependency Confusion or Maintainer Hijacking (e.g., the event-stream or xz-utils compromises)—leverage these legitimate delivery channels [5].

The vulnerability is rooted in **ambient authority** [6]. In standard POSIX-compliant language runtimes (Node.js, Python), an imported dependency operates with the exact privilege matrix of the host application. A micro-package imported exclusively to pad a string inherently possesses the authorization to read system environment variables (exfiltrating cryptographic keys), access the file system, and establish outbound Transmission Control Protocol (TCP)

sockets.
Furthermore, lifecycle scripts (e.g., postinstall) execute arbitrary Turing-complete code at the exact moment of ingestion, preempting explicit invocation by the developer. Because of Rice's Theorem, establishing the non-malicious intent of arbitrary Turing-complete code via static analysis is mathematically undecidable. The cryptographically secure perimeter flawlessly delivers a malicious payload directly into an undefended execution space.

IV. Statistical Opacity: The Epistemology of AI Parameter Spaces

If the procedural software ecosystem suffers from unverified deterministic execution, Large Language Models (LLMs) introduce an exponentially more dangerous paradigm: the semantic opacity of high-dimensional neural network weights.

A. The Ontology of Tensors: What Do Model Weights Encode?

To understand AI vulnerabilities, we must establish the ontological nature of model weights. Unlike deterministic software containing discrete Boolean logic paths, LLMs are parameterized functions $f(x; \theta)$, optimized via stochastic gradient descent (SGD).
When deploying an open-weights model, one downloads a multi-dimensional matrix of billions of floating-point numbers ($\theta \in \mathbb{R}^d$). In a Transformer architecture [7], these weights function as mathematical routing protocols. They parameterize:

1. **Attention Matrices (W_Q, W_K, W_V):** These project input tokens into geometric spaces, calculating inner products to resolve syntactic dependencies and semantic context across a sequence.
2. **Multi-Layer Perceptrons (MLPs):** These act as continuous associative memory banks, projecting complex latent patterns to recall factual relationships [8].

Weights do not encode discrete logic (if x == "hack"). They encode concepts strictly as topological directions in a continuous manifold. The weights geometrically deform input tensors through hundreds of dimensional layers to minimize the cross-entropy loss of predicting the next statistical token.

B. The Epistemic Crisis: "How Do You Even Know?"

The core of the AI security crisis is epistemological: *We mathematically cannot prove what the weights will do in novel contexts prior to execution.*
Due to the computational pressures of SGD, models learn to represent exponentially more features than they have dimensions (neurons), leading to a phenomenon known as **superposition** [9]. Consequently, individual neurons are **polysemantic**—a single neuron may activate simultaneously for the concept of "cryptography," the Arabic language, and HTML formatting, depending on the linear combination of surrounding activations.
Because of polysemanticity, traditional empirical verification ("black-box" benchmarking) structurally fails. Evaluating outputs y given inputs x cannot mathematically prove the absence of a hidden, malicious manifold deep within the latent space.

C. Injection Vectors and Latent Sleeper Agents

This profound epistemic opacity facilitates devastating takeover mechanisms:

1. **Sleeper Agents (Semantic Backdoors):** Hubinger et al. (2024) demonstrated that adversaries can fine-tune highly capable models to harbor latent backdoors [10]. The model exhibits perfectly benign behavior during perimeter benchmarking. However, when triggered by a specific cryptographic string or syntactic pattern, the weights geometrically reroute the computation, causing the model to output exploitative code. Standard alignment techniques (like Reinforcement Learning from Human Feedback, RLHF) structurally fail to eliminate these backdoors, as the hidden trigger remains unactivated during safety training [10].
2. **Prompt Injection:** The execution boundary of an LLM mirrors a fundamentally insecure von Neumann architecture, where system instructions and user data share the exact same memory space (the context window). The "squishy inside" cannot definitively differentiate between benign data and adversarial execution commands, allowing the payload itself to hijack the control flow [11].

V. Cryptographic Provenance: The DNSSEC Bootstrapping Dilemma

This crisis of transitive trust maps flawlessly to Public Key Infrastructure (PKI), specifically the DNSSEC key distribution bottleneck [12]. DNSSEC mitigates BGP hijacking and DNS spoofing by establishing a cryptographic chain of trust from the Root Zone (.) down to individual domains.

A. Hierarchical Fragility and RFC 5011

DNSSEC pairs mathematically unassailable perimeter cryptography with a deeply fragile, human-in-the-loop internal distribution mechanism.

- **The Hard Perimeter:** Cryptographic signatures (RSA/ECDSA) ensure transit integrity. Assuming $P \neq NP$, tampering is computationally infeasible.
- **The Squishy Interior:** The entire global hierarchy relies on an initial Trust Anchor—the Root Zone Key Signing Key (KSK). How does a computational resolver prove the *initial* key is legitimate? It relies on immutable configuration files baked into operating systems or out-of-band updates (formalized in RFC 5011) [13].

This constitutes the PKI bootstrapping problem. If an adversary compromises the initial bootstrapping mechanism or coercions a downstream Top-Level Domain (TLD) registry, the cryptography *weaponizes itself against the defender*. The resolver uses mathematically perfect logic to confidently validate maliciously signed routing data. We rigorously verify the mathematical provenance of the signature, but we lack mechanisms to verify the semantic truth of the underlying administrative key.

VI. Systemic Mitigations: Hardening the Computational Core

To survive in an era of hyper-composable abstractions, security architectures must abandon perimeter-only provenance in favor of structural constraint and Continuous Semantic Verification.

A. Object-Capability Security (Resolving Procedural Transitivity)

Language runtimes must deprecate ambient authority in favor of Object-Capability (OCap) Models [14]. Modern execution environments like WebAssembly (Wasm) and Deno mandate an

explicit "deny-by-default" architecture. Dependencies must explicitly declare, and be granted, unforgeable capability handles by the host (e.g., --allow-net=api.github.com). If a transient math library attempts an unauthorized system call, the runtime mathematically traps the instruction. This collapses the transitive trust chain by functionally isolating the interior.

B. Mechanistic Interpretability and zk-ML (Resolving Statistical Opacity)

To secure neural parameter spaces, AI must transition from an empirical black box to a verifiable glass box.

1. **Zero-Knowledge Provenance (zk-ML):** Utilizing Zero-Knowledge Machine Learning (zk-SNARKs), model creators must provide mathematical proofs asserting that a specific matrix θ was derived *exclusively* from an audited dataset [15]. This structurally prevents post-training backdoor injection without rendering the proof invalid.
2. **Mechanistic Interpretability:** We must pioneer the reverse-engineering of latent tensor spaces. Utilizing **Sparse Autoencoders (SAEs)**, researchers are actively disentangling polysemantic matrices into human-interpretable, monosemantic "feature dictionaries" [16]. Future "Model Antivirus" will perform static analysis directly on the model's geometry, projecting weights through an SAE to algorithmically detect "deception circuits" prior to inference deployment.

C. Decentralized Threshold Consensus (Resolving Cryptographic Bootstrapping)

Replacing monolithic hierarchical Trust Anchors requires Decentralized Threshold Verification. Utilizing Transparency Logs (structurally akin to Certificate Transparency for X.509 [17]) and Multi-Party Computation (MPC), cryptographic bootstrapping must rely on a continuously audited, globally distributed consensus ledger. This ensures no single initial localized compromise can unilaterally dictate the transitive routing reality of the network.

VII. Conclusion

The "hard on the outside, soft and squishy on the inside" paradigm constitutes the preeminent systemic risk to modern computational infrastructure. We have engineered adamantine cryptographic perimeters, only to voluntarily usher unauditable, immensely powerful Trojan Horses through the front gates. Whether observing a transitive NPM script executing with root privileges, a statistically poisoned neural network hiding a mathematical sleeper agent, or a rogue DNSSEC trust anchor confidently routing traffic to a sinkhole, the etiology is identical: our architectures validate the wrapper but implicitly trust the execution of the payload.

Securing the next epoch of computing requires a profound ontological shift. We must view nested dependencies, multi-dimensional parameter matrices, and root cryptographic keys not as intrinsically trusted artifacts, but as mathematically suspect internal states requiring strict OCap isolation and continuous mechanistic interrogation.

References

[1] W. R. Cheswick and S. M. Bellovin, *Firewalls and Internet Security: Repelling the Wily Hacker*. Addison-Wesley, 1994.

[2] K. Thompson, "Reflections on trusting trust," *Communications of the ACM*, vol. 27, no. 8, pp.

761-763, 1984.
[3] M. Zimmermann, C. Dietrich, C. Staicu, and M. Pradel, "Small World with High Risks: A Study of Security Threats in the npm Ecosystem," in *28th USENIX Security Symposium*, 2019, pp. 995-1010.
[4] L. Santiago, A. L. N. Reddy, and T. Ristenpart, "Sigstore: Software Signing for Everybody," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021.
[5] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020, pp. 23-43.
[6] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278-1308, 1975.
[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[8] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer Feed-Forward Layers Are Key-Value Memories," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
[9] N. Elhage et al., "Toy Models of Superposition," *Anthropic Transformer Circuits Thread*, 2022. [Online]. Available: https://transformer-circuits.pub/2022/toy_model/index.html
[10] E. Hubinger et al., "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training," *arXiv preprint arXiv:2401.05566*, 2024.
[11] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
[12] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, Internet Engineering Task Force, 2005.
[13] M. StJohns, "Automated Updates of DNS Security (DNSSEC) Trust Anchors," RFC 5011, Internet Engineering Task Force, 2007.
[14] M. S. Miller, "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control," Ph.D. Dissertation, Johns Hopkins University, Baltimore, MD, 2006.
[15] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun, "Scaling up Trustless Execution of Machine Learning Models," *Cryptology ePrint Archive*, Paper 2022/1045, 2022.
[16] T. Bricken et al., "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," *Anthropic Transformer Circuits Thread*, 2023.
[17] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, Internet Engineering Task Force, 2013.